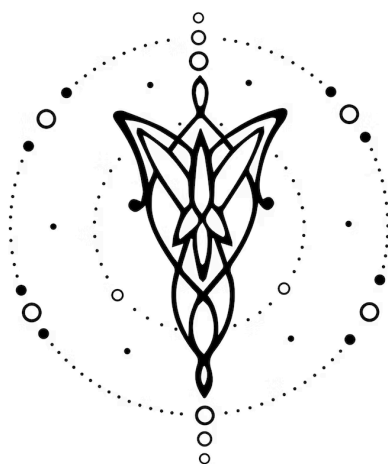




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Lucila	Borinsky	63039	lborkinsky@itba.edu.ar
Santiago Diaz	Sieiro	63058	sdiazsieiro@itba.edu.ar
Axel Armando	Farina	63081	axfarina@itba.edu.ar
Hwa Pyoung	Kim	62129	hkim@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [TLA PLOTTER](#).

3. Dominio

Desarrollar un lenguaje declarativo que permite crear **escenas 2D** mediante la descripción de figuras geométricas simples como rectángulos, círculos, líneas, elipses, polilíneas y polígonos, acompañadas de propiedades de estilo y transformaciones. El objetivo es permitir la creación de representaciones visuales de manera sencilla y eficiente, donde el usuario pueda definir escenas completas especificando sus componentes y propiedades a través de parámetros fáciles de entender.

El lenguaje debe ser intuitivo y de fácil uso, permitiendo a los usuarios declarar las formas y sus propiedades de manera directa. Para cada tipo de forma, se deben especificar parámetros como ubicación (at), tamaño (size), color (fill y stroke), y transformaciones (como translate, rotate y scale), de forma que el programa genere un archivo de salida en formato SVG (Scalable Vector Graphics), el cual puede visualizarse fácilmente en cualquier navegador web.

Además, el diseño del lenguaje debe permitir la organización por capas (layer), el agrupamiento de elementos (group) y la reutilización de figuras a través de símbolos y su posterior uso con el comando use. Esta flexibilidad asegura que los elementos visuales puedan definirse una vez y reutilizarse múltiples veces dentro de la escena.

El objetivo final es proporcionar una herramienta que permita a los usuarios crear escenas visuales de manera rápida, eficiente y clara, sin necesidad de tener conocimientos avanzados de diseño gráfico o de programación compleja.

4. Construcciones

El lenguaje desarrollado ofrecerá las siguientes construcciones, prestaciones y funcionalidades:

- (I). **Escenas:** Se podrá crear una o varias escenas en un lienzo 2D, especificando el tamaño y el fondo.
- (II). **Figuras geométricas:** Se podrán crear figuras geométricas como rectángulos, círculos, líneas, elipses, polígonos y polilíneas, indicando sus propiedades (como ubicación, tamaño, color, etc.).
- (III). **Estilo:** El lenguaje permitirá asignar propiedades de estilo a las figuras, como relleno, trazo, grosor del trazo, opacidad, y estilos de línea
- (IV). **Transformaciones:** Las formas podrán ser transformadas con operaciones de traslación, rotación y escala, tanto a nivel individual como grupal.
- (V). **Capas:** Se podrá organizar las figuras en capas, con un control sobre el orden de pintado mediante la propiedad z-index.
- (VI). **Grupos:** Las formas podrán agruparse para ser transformadas en conjunto utilizando la construcción de grupo.

- (VII). **Símbolos y reutilización:** El lenguaje permitirá definir símbolos reutilizables que podrán ser instanciados múltiples veces en la escena con distintas posiciones y transformaciones.
- (VIII). **Paleta de colores:** Se podrá definir una paleta de colores con nombres específicos, facilitando el uso consistente de colores en todo el proyecto.
- (IX). **Variables y expresiones:** El lenguaje soportará la definición de variables y permitirá realizar operaciones matemáticas básicas sobre ellas, facilitando la manipulación de dimensiones y posiciones.
- (X). **Metadatos y exportación:** El lenguaje permitirá la exportación a SVG, asegurando que el archivo generado sea válido y contenga metadatos como title y description.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que construye un rectangle “casa” y un circle “sol” a través de una escena.
- (II). Un programa que construya distintos rectangle con distintos valores de strokeWidth, lineJoin y opacity.
- (III). Un programa que declare distintas paletas de colores y las use dentro de fill.
- (IV). Un programa que traslade y rote 2 figuras dentro de un group.
- (V). Un programa que declare un símbolo y lo use en varias posiciones con escalas distintas.
- (VI). Un programa que declare capas diferentes.
- (VII). Un programa que superpone una figura sobre otra en la misma capa mediante “z”.
- (VIII). Un programa que dibuje una línea diagonal con stroke y strokeWidth definidos.
- (IX). Un programa que construya una elipse con radios y opacidades distintas.
- (X). Un programa que utilice variables para construir y declarar tamaños de figuras.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa mal formado.
- (II). Un programa que declare figuras sin sus respectivos parámetros de tamaño (size o radius).
- (III). Un programa que dibuje una figura sin el parámetro de posición (at).
- (IV). Un programa que dibuje un objeto por fuera de la escena declarada.
- (V). Un programa que asigne a parámetros valores fuera de su dominio (tamaños negativos, colores donde sus valores superan el valor 255).
- (VI). Un programa que declare dos ids idénticos de simbolos o capas.
- (VII). Un programa que utilice objetos sin haber sido previamente declarados como símbolos.
- (VIII). Un programa que declare parámetros que no pertenecen a las figuras las cuales se desean dibujar.

6. Ejemplos

Crear una escena de **800×600** con fondo gris; dibujar una **casa** (rectángulo) y un **sol** (círculo), asegurando que el sol quede por encima mediante $z = 10$.

```
// Escena mínima (casa + sol)
scene "Mi Dibujo" size(800, 600) {
  background color(240,240,240);

  // Casa
  draw rectangle with {
    id "house";
    at(200,300); size(200,150);
    fill color(200,150,100); stroke color(80,60,40); strokeWidth 2;
    layer "background";
  }

  // Sol
  draw circle with {
    id "sun";
    at(300,100); radius 50;
    fill color(255,255,0);
    layer "sprites"; z 10;
  }
}
```

Definir un símbolo reutilizable **tree**, declarar una paleta de colores, crear capas **background** y **sprites**, y reutilizar el símbolo tres veces con distintas posiciones y escalas

```
// Símbolo reutilizable (árbol), capas y 'use'
scene "Parque" size(900, 400) {
  palette { "trunk": #6B4F2A; "crown": #2E8B57; "grass": #4CAF50; }
  layer "background"; layer "sprites";

  symbol "tree" {
    draw rectangle with { at(0,30); size(20,40); fill "trunk"; }
    draw circle with { at(10,20); radius 25; fill "crown"; }
  }

  draw rectangle with { at(0,350); size(900,50); fill "grass"; layer
"background"; }

  use "tree" at(120,320) with { layer "sprites"; }
  use "tree" at(180,330) with { scale(0.8); }
```

```
use "tree" at(260,310) with { scale(1.2); }  
}
```

Mostrar un caso de **rechazo**. Intentar usar **size** y **fill** en una línea, cuando las propiedades válidas son **from/to** y **stroke**:

```
// Ejemplo de rechazo: propiedades inválidas para 'line'  
scene "Error" size(400,300) {  
  draw line with {  
    size(100,20); // ERROR: 'line' solo admite 'from' y 'to'  
    fill #000000; // ERROR (según diseño): las líneas usan 'stroke'  
  }  
}
```